

13c — Kubernetes (K8s)

Key Concepts

- **Kubernetes** = Container orchestration platform — manages containers at scale
- Docker runs containers; Kubernetes manages fleets of containers
- Solves: self-healing, auto-scaling, rolling updates, load balancing, service discovery
- Abbreviated as **K8s** (8 letters between K and s)

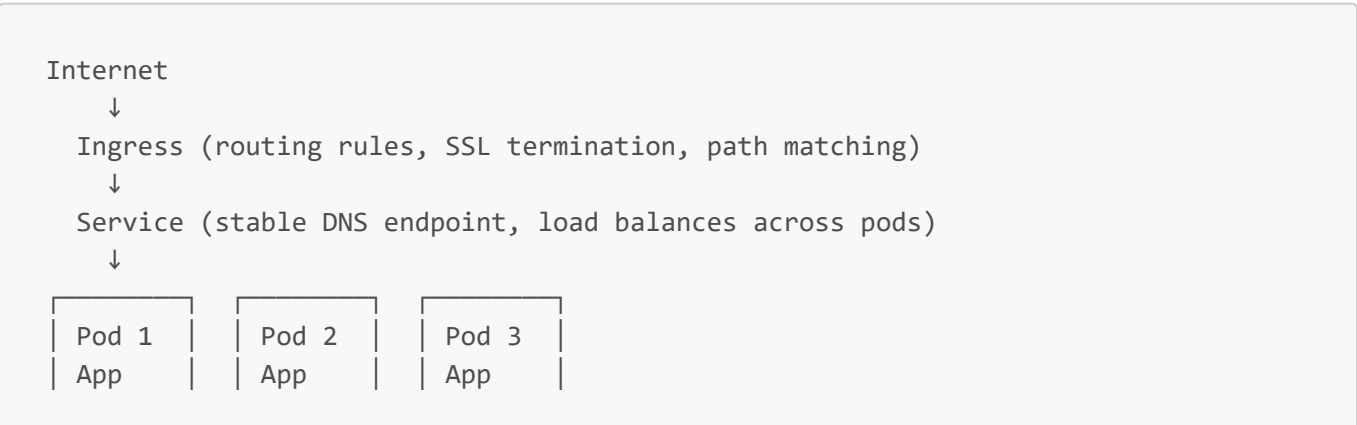
Core Building Blocks

Concept	Definition
Pod	Smallest deployable unit — wraps one or more containers sharing network and storage
Node	Physical or virtual machine that runs Pods
Cluster	Full K8s environment — set of nodes managed together
Deployment	Defines how many Pod replicas to run, manages rolling updates
Service	Stable DNS name/fixed endpoint that routes to healthy Pods
Ingress	External HTTP/HTTPS entry point — routing rules, SSL termination

Cluster Structure



Full Traffic Flow



managed by Deployment

Deployment Example

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-api
spec:
  replicas: 3           # always keep 3 pods running
  template:
    spec:
      containers:
        - name: api
          image: myapp:latest
```

If a Pod crashes → Deployment automatically replaces it.

Service Types

Type	Use case
ClusterIP	Internal communication only (default)
NodePort	Expose on a port on each node
LoadBalancer	External traffic — creates a cloud load balancer

Why Services exist: Pods are ephemeral — they die and get new IPs. Service provides a stable DNS name that always routes to healthy Pods.

Key Capabilities

Capability	What K8s does
Self-healing	Pod crashes → K8s restarts it automatically
Auto-scaling (HPA)	Traffic spikes → K8s adds more pods
Rolling updates	Deploy new version gradually — zero downtime
Rollback	New version breaks → roll back instantly
Load balancing	Distributes traffic across all healthy pods
Service discovery	Pods find each other by DNS name, not IP

ConfigMap vs Secret

Resource	Use for
ConfigMap	Non-sensitive config — feature flags, URLs, settings
Secret	Sensitive data — passwords, API keys, connection strings (stored encrypted)

Industry Examples

Company	Kubernetes Usage
Google	Invented K8s — runs billions of containers weekly internally
Spotify	Deploys 10M+ containers per day via K8s
Airbnb	Auto-scales during peak booking periods
Xbox	Uses Azure Kubernetes Service (AKS) for game backend services

Interview Questions

Q1: What is Kubernetes and what problem does it solve?

Container orchestration platform. Docker runs containers — K8s manages fleets at scale: self-healing, auto-scaling, rolling updates, load balancing.

Q2: What is a Pod?

Smallest deployable unit in K8s. Wraps one or more containers sharing network and storage. You never run containers directly — you run Pods.

Q3: What is a Deployment?

Defines how many Pod replicas to run and manages rolling updates. If a Pod crashes, Deployment automatically replaces it.

Q4: Why do we need Services if we already have Pods?

Pods are ephemeral — they die and get new IPs. Service provides a stable DNS name and fixed endpoint routing to healthy Pods regardless of which instances are running.

Q5: Service vs Ingress?

Service = stable internal endpoint routing to Pods. Ingress = external HTTP/HTTPS entry point — handles routing rules, path matching, SSL termination before traffic reaches a Service.

Q6: ConfigMap vs Secret?

ConfigMap = non-sensitive config injected into Pods. Secret = sensitive data stored encrypted (passwords, API keys).